Travel Code REST API · Reference

Reference · Authentication, Search, Orders, Wallet

> *Complete reference for the Travel Code public API: authentication, location lookups, multi-service search (hotels, flights, trains, transfers), and the order/booking lifecycle.*

Base URL: `https://api.travel-code.com` **Format:** JSON, except `POST /v1/search/hotels/stream` which uses Server-Sent Events (`text/event-stream`). **Versioning:** All endpoints are exposed under `/v1/...`. Breaking changes ship as `v2` with a 6-month deprecation window.

1. Overview

This document describes every public endpoint of the Travel Code API. The API is grouped into five concerns:

1. **Authentication** — `/v1/auth/*` (JWT) and `/oauth/*` (OAuth 2.0 + PKCE for AI agents and MCP).
2. **Data lookups** — `/v1/data/...` autocompletes for airports, hotel locations, train stations, etc.
3. **Search** — `/v1/search/{hotels|flights|trains|transfers}/...` for each service type.
4. **Orders** — `/v1/orders/...` covers the full booking lifecycle for every service type.
5. **Wallet** — `/v1/wallet/...` for deposit-funded corporate accounts.

Cache lifetimes

Every multi-supplier search step caches its result for **15 minutes**. After expiration the search must be re-run before the result can be used downstream (e.g. before requesting offers or creating an order).

Authentication tiers

Tier	Use case	Header
None	Hotel/flight search, location autocompletes, public read endpoints.	—
JWT	Server-to-server traffic for a single integrator account.	`Authorization: Bearer <accessToken>`
OAuth 2.0	Third-party AI agents and MCP servers acting on behalf of an end user. Authorization-code with PKCE.	`Authorization: Bearer <accessToken>`

User-scoped endpoints (orders, wallet, modify, cancel) require a token. Anonymous price discovery (hotel/flight search) does not.

2. Authentication

2.1 Login

```
POST /v1/auth/login
```

Exchanges credentials for an **access token** and (optionally) embeds the user profile.

Parameter	Type	Req	Description
`email`	string	yes	Account email.
`password`	string	yes	Account password.
`embedded`	string[]	no	Side-loads. Currently supported: `user`.

Response

```
{
  "accessToken": "eyJhbGciOiJI...",
  "expiresAt":   "2026-05-01T15:42:00+00:00",
  "user": {
    "id":          1234,
    "username":    "ops@example.com",
    "role":        "agent",
    "userType":    "company",
    "language":    "en",
    "phoneNumber": "+44..."
  }
}
```

`user` is included only when `embedded` contains `user`.

2.2 Refresh JWT

```
POST /v1/auth/jwt/refresh
```

Rotates the refresh token and returns a fresh access token. Call **before** the current `accessToken` expires.

Parameter	Type	Req	Description
`refreshToken`	string	yes	Refresh token issued by `/v1/auth/login`.

Response

```
{
  "accessToken":  "eyJhbGciOiJI...",
  "refreshToken": "eyJhbGciOiJI...",
  "expiresIn":    3600,
  "tokenType":    "Bearer"
}
```

2.3 Refresh legacy access token

```
PUT /v1/auth/refresh
Authorization: Bearer <expiring-or-fresh-token>
```

For accounts still issued non-JWT tokens. Re-issues an access token using the existing one as proof of identity. New integrations should use §2.2.

2.4 OAuth 2.0 (third-party clients)

For AI agents, MCP servers, and any client acting on behalf of an end user, Travel Code exposes a standards-compliant OAuth 2.0 authorization-code flow with PKCE.

2.4.1 Discovery

```
GET /.well-known/oauth-authorization-server
```

Returns RFC 8414 metadata so clients can auto-configure:

```
{
  "issuer":                  "https://api.travel-code.com",
  "authorization_endpoint":  "https://api.travel-code.com/oauth/authorize",
  "token_endpoint":          "https://api.travel-code.com/oauth/token",
  "registration_endpoint":   "https://api.travel-code.com/oauth/register",
  "revocation_endpoint":     "https://api.travel-code.com/oauth/revoke",
  "scopes_supported":        ["flights:search", "flights:status", "flights:stats", "airports:read",
       "airlines:read"],
  "response_types_supported":["code"],
  "grant_types_supported":   ["authorization_code", "refresh_token"],
  "token_endpoint_auth_methods_supported": ["none"],
  "code_challenge_methods_supported":      ["S256"]
}
```

2.4.2 Dynamic client registration

```
POST /oauth/register
Content-Type: application/json

{
  "client_name":   "My MCP server",
  "redirect_uris": ["https://example.com/callback"]
}
```

Returns `{ "client_id": "...", "client_secret_expires_at": 0 }`. Public clients (PKCE-only) are supported — no `client_secret`.

2.4.3 Authorize → Token

Standard authorization-code-with-PKCE flow:

1. Redirect the user to `GET /oauth/authorize?response_type=code&client_id=...&redirect_uri=...&scope=...&code_challenge=...&code_challenge_method=S256`.
2. After user consent, the user-agent is redirected back with `?code=...`.
3. Exchange the code for tokens:

```
POST /oauth/token
Content-Type: application/x-www-form-urlencoded

grant_type=authorization_code
&code=...
&redirect_uri=...
&client_id=...
&code_verifier=...
```

Response is the standard `{ access_token, refresh_token, token_type, expires_in, scope }`.

2.4.4 Revoke

```
POST /oauth/revoke
Content-Type: application/x-www-form-urlencoded

token=<access_or_refresh_token>
&client_id=...
```

2.5 Using the bearer token

All authenticated endpoints accept the same header regardless of how the token was obtained:

```
Authorization: Bearer <accessToken>
```

3. Booking flows

Every booking type follows a similar pattern: resolve location → run a search → pick an offer → create an order → optionally cancel/modify. The exact endpoints differ per service.

Hotel pipeline

```
1) /v1/data/hotel-locations            -> resolve locationId or hotelId
2) /v1/search/hotels/stream            -> hotel cards with cheapest offer (SSE)
   /v1/search/hotels/list              -> page of cards from cache
   /v1/search/hotels/map               -> minimal geo points from cache
   /v1/search/hotels/search-names      -> filter cached results by hotel name
3) /v1/search/hotels/offers            -> all rate plans for one hotel + content data
4) POST /v1/orders                      -> create the booking
```

Step 2 takes a `locationId` (city/region — positive id) or a single `hotelId` (negative giataId) from the step 1 response and returns hotels with the cheapest available offer. Of the search routes, only `/v1/search/hotels/stream` performs the actual search; `/list`, `/map`, and `/search-names` are cache-only — they read from the cache populated by `stream` and must be called within its 15-minute TTL.

Flight pipeline

```
1) /v1/data/airports                   -> IATA codes
2) POST /v1/search/flights             -> kick off async search, returns cacheId
3) GET  /v1/search/flights/{cacheId}   -> read the cached result page
4) POST /v1/orders                      -> create the booking
```

Two specialty searches sit on top of the same engine:

```
POST /v1/search/flights/calendar          -> price grid for a date range
POST /v1/search/flights/comparison-table  -> side-by-side fare-class comparison
```

Train pipeline

```
1) /v1/data/railway-stations           -> station codes
2) POST /v1/search/trains              -> kick off async search
3) GET  /v1/search/trains/{cacheId}    -> read result
4) POST /v1/orders                      -> create the booking
```

Transfer pipeline

```
1) /v1/data/locations-autocomplete     -> resolve Google Place IDs
2) POST /v1/search/transfers           -> synchronous search
3) POST /v1/orders                      -> create the booking
```

Transfers are synchronous — there is no cache step.

4. Data lookups

Read-only endpoints for autocompletes and code resolution. All are unauthenticated.

Endpoint	Purpose
`GET /v1/data/hotel-locations`	Hotel-search locations (cities, regions, airports, hotels).
`GET /v1/data/airports`	Airports (IATA + city + country).
`GET /v1/data/airlines`	Airlines (IATA + name).
`GET /v1/data/railway-stations`	Train stations.
`GET /v1/data/locations-autocomplete?q=...`	Google-Places-backed location autocomplete (used for transfers).
`GET /v1/data/google-place-details?id=...`	Resolve a Google Place ID to coordinates / address.
`GET /v1/data/countries`	Countries (ISO + nationality).
`GET /v1/data/hotels/filters`	Available hotel filter values per market.
`GET /v1/data/vocabulary`	Translated UI strings (boards, property types, cabin classes).

4.1 Hotel locations

```
GET /v1/data/hotel-locations
```

Search city, region, or hotel locations. Use `children[].id` as `location` in subsequent search calls.

Parameter	Type	Req	Description
`search`	string	yes	Query string (Cyrillic or Latin).
`limit`	int	no	Max items (default 15).
`scope`	string	no	`onlyNames` to return only names.

Response

```
{
  "items": [
    {
      "type": "regions",
      "text": "Regions",
      "children": [
        {"id":  624596, "name": "Tbilisi", "address": "Georgia", "countryCode": "GE"},
        {"id":  886591, "name": "Tbilisi International Airport – TBS", "address": "Georgia,
         Tbilisi", "countryCode": "GE"}
      ]
    },
    {
      "type": "hotels",
      "text": "Hotels",
      "children": [
        {"id": "-449173", "name": "Tbilisi House", "address": "Georgia, Tbilisi, Nasakirali Street
         14",  "countryCode": "GE"},
        {"id": "-542499", "name": "Tbilisi Inn",   "address": "Georgia, Tbilisi, Metekhi street,
         20",     "countryCode": "GE"}
      ]
    }
  ],
  "time": 0.51
}
```


Semantics for `location` in subsequent search calls:

- positive id (`12345`) — city or region search.
- negative id (`-67890`) — single hotel search by giataId.

5. Search — Hotels

Four endpoints share the same parameter set; they differ only in the shape of the response.

5.1 Common parameters

Parameter	Type	Req	Description
`location`	int / string	yes	Location id from `/v1/data/hotel-locations`. Positive id = city/region; negative id = single hotel by giataId.
`checkin`	date	yes	`YYYY-MM-DD`.
`checkout`	date	yes	Same. Stay length capped at 28 nights, otherwise `checkin` is back-shifted.
`countryCode`	string	yes	Guest nationality ISO code (e.g. `UK`).
`guests`	array	yes	Per-room occupancy, see §5.2.
`offset`	int	stream/list	Pagination offset.
`limit`	int	stream/list	Page size.
`sort`	string	no	`recommend` (default) or `price`.
`filter`	object	no	Filter object, see §5.3.
`id`	int[] / int	list	Restrict result to a set of giataIds (`/list` only).
`mapBoundsSwLat`	float	no	Southwest corner latitude. When all four `mapBounds*` are present, the search runs in viewport mode. e.g. `41.3431`
`mapBoundsSwLng`	float	no	Southwest corner longitude. e.g. `2.0695`
`mapBoundsNeLat`	float	no	Northeast corner latitude. e.g. `41.4696`

Parameter	Type	Req	Description
`mapBoundsNeLng`	float	no	Northeast corner longitude. e.g. `2.2280`

5.2 `guests` — per-room occupancy

```
[
  {"adults": 2},                                          // 1 room, 2 adults
  {"adults": 2, "children": 1, "childrenAges": [5]},      // 1 room, 2A + 1 child age 5
  {"adults": 2, "children": 2, "childrenAges": [3, 7]}    // 2 children, ages 3 & 7
]
```

Parameter	Type	Req	Description
`adults`	int	yes	1–4.
`children`	int	no	0–3.
`childrenAges`	int[]	when `children > 0`	Each value 0–17.

Multiple rooms are passed as multiple objects in the array. Total `adults + children` count is propagated to all suppliers; child ages are part of the search hash and must match exactly when used downstream.

5.3 `filter` — filter object

Only the fields you want to apply need to be sent.

Parameter	Type	Req	Description
`minPrice`	int	no	Minimum nightly price in site currency, converted to USD server-side. `0` is treated as null.
`maxPrice`	int	no	Maximum nightly price (site currency, converted).
`starRating[]`	int[]	no	One or more of `1`, `2`, `3`, `4`, `5`.
`boards[]`	string[]	no	Meal plan codes — see §5.4.
`payments`	string / string[]	no	`full_refund` to require a fully refundable rate.
`propertyTypes[]`	string[]	no	Codes — see §5.5.

```
"filter": {
  "minPrice": 50,
  "maxPrice": 300,
  "starRating[]": [4, 5],
  "boards[]": ["AI", "FB"],
  "payments": ["full_refund"],
  "propertyTypes[]": ["hotel", "resort"]
}
```

5.4 Board codes

Code	Meaning
`RO`	Room only
`BI`	Breakfast included
`LI`	Lunch included
`DI`	Dinner included
`HB`	Half board
`FB`	Full board
`AI`	All inclusive

5.5 Property types

```
hotel, aparthotel, apartment, hostel, guesthouse, bed_and_breakfast,
resort, villa, motel, bungalow, inn, country_house, holiday_park,
camping, boutique, capsule, specialty
```

5.6 Stream search (SSE)

```
POST /v1/search/hotels/stream
```

Streams hotel cards as they arrive from the supplier fan-out.

- `Content-Type: text/event-stream`
- Server timeout 120 s — set client `--max-time 130`.
- Always pair with `Cache-Control: no-cache` on intermediaries.

Required body fields: `countryCode`, `guests`, `offset`, `limit`, `location`, `checkin`, `checkout`. Optional: any field from §5.1, plus `sort` and `filter`.

Event types

Event	When	Payload
`connected`	First event	`{"status": "ok"}` (`"cached": true` if cache hit)
`hotels`	Recommend mode	Batch: `{"batch": int, "count": int, "hotels": [...]}`
`sorted_hotels`	Recommend mode	Sorted batch: `{"count": int, "total": int, "chunk": int, "hotels": [...]}`
`count`	Progress	`{"batch": int, "count": int, "total": int}`
`completed`	End of stream	`{"count": int, "hotels": [...], "cacheKey": "{unified}:filter:..."}`
`error`	Failure	`{"message": "..."}`
`timeout`	120 s exceeded	`{"message": "Stream timeout"}`

In `recommend` mode hotels arrive via `hotels` / `sorted_hotels` and `completed.hotels` is empty. In `price` mode only `count` events stream and `completed` carries the sorted, paginated page.

`cacheKey` from `completed` is used downstream by `/search-names` to filter the cached result by hotel name.

5.7 List

```
POST /v1/search/hotels/list
```

Reads a JSON page of hotel cards from the cache populated by `/v1/search/hotels/stream`. `/list` does **not** run a new search — it must be called after `stream` and within the 15-minute cache TTL. The body is identical to the stream endpoint, plus a required `id` (one or more giataIds) field that scopes the result.

Response


```
{
  "count": 424,
  "hotels": [
    {
      "id": 1431950,
      "price": 91,
      "total": 181,
      "refundable": true,
      "hotelNames": {"mainName": "ibis Styles Old Tbilisi", "alsoKnownNames": ""},
      "propertyName": "Ibis Styles Old Tbilisi",
      "heroImage": "https://i.travelapi.com/lodging/.../a6f3c23b_b.jpg",
      "heroImages": ["https://i.travelapi.com/lodging/.../a6f3c23b_b.jpg", "..."],
      "description": "In Tbilisi (Tbilisi City Centre)",
      "ratings": {"count": 6, "overall": "4.2", "reviews": "reviews", "text": "excellent"},
      "starRating": 4,
      "longitude": "44.80655043",
      "latitude": "41.69122488",
      "meal": "Breakfast included",
      "availableRooms": null
    }
  ]
}
```


5.8 Map

```
POST /v1/search/hotels/map
```

Minimal geo points for map rendering. Returns one entry per hotel with `id`, `latitude`, `longitude`, `price` (integer in site currency).

Like `/list`, this endpoint reads from the cache populated by `/v1/search/hotels/stream` and does **not** perform a search of its own. It must be called after `stream` and within the 15-minute cache TTL.

```
{
  "count": 142,
  "hotels": [
    {"id": 1431950, "latitude": 41.6912, "longitude": 44.8066, "price": 91}
  ]
}
```

The `mapBoundsSwLat` / `SwLng` / `NeLat` / `NeLng` quad is the canonical way to scope `/map` (and `/list`) to the visible area.

5.9 Search names

```
GET /v1/search/hotels/search-names?ck=...&q=...
```

Filter the cached search by hotel name. `ck` is the `cacheKey` returned in `completed` (must match `^{unified}:(stream:prices|filter):...`). Returns up to 100 hits sorted by relevance.

```
[
  {"id": 1431950, "name": "Ibis Styles Old Tbilisi", "score": 0.93}
]
```

5.10 Single-hotel offers

```
POST /v1/search/hotels/offers
```

Returns all available rate plans for a single hotel. The response carries a `sessionId` for the cached result and, for each rate, an `id` that identifies it downstream when creating an order.

Alongside the rate plans, the response also returns the hotel content (property context) — name, star rating, address, images, description, and geo coordinates — so a separate content lookup is not required.

Parameter	Type	Req	Description
`id`	int	yes	Hotel giataId.
`checkin`	date	yes	`YYYY-MM-DD`.
`checkout`	date	yes	`YYYY-MM-DD`.
`countryCode`	string	yes	Guest nationality ISO code.
`guests`	array	yes	Same shape as §5.2.
`location`	int	no	Same location as the prior `stream` call — enables cache reuse.

Response (excerpt)


```
{
  "sessionId": "69f38317223480.46921054",
  "property": {
    "id": "69923287",
    "address": "Georgia, Tiflis, Kote Abkhazi 39",
    "name": "Ibis Styles Old Tbilisi",
    "heroImage": "https://api-img.hotelston.com/.../958699859.jpg",
    "images": [{"url": "https://api-img.hotelston.com/.../958699859.jpg"}],
    "starRating": 4,
    "longitude": 44.80655170000001,
    "latitude":  41.6912095,
    "checkin":  {"minAge": 18, "beginTime": "15:00", "endTime": null},
    "checkout": {"beginTime": "12:00", "endTime": null, "time": null},
    "description": [
      {"title": "General",  "text": "<p><strong>...</strong></p>"},
      {"title": "Location", "text": "<p><strong>...</strong></p>"}
    ],
    "phone": "+995322002022",
    "city": "Tiflis",
    "country": "Georgia"
  },
  "offers": {
    "Standard Double Room": {
      "content": {"area": "17.0", "views": null, "photos": ["..."]},
      "rates": [
        {
          "id": "5e266508-dee21a18-6599dbce-9dabfbe3",
          "hotelCode": "1431950",
          "price": {
            "currency": "USD",
            "total": 181.2, "nights": 2, "rooms": 1,
            "nightly": 90.6, "extra": 0, "totalWithExtra": 181.2,
            "deposit": null
          },
          "cancelPolicy": {
            "refundable": true,
            "rules": [
              {"deadline": "2026-06-14 11:00:00", "type": "amount", "value": 75},
              {"deadline": "2026-06-15 00:00:00", "type": "amount", "value": 151}
            ],
            "title": "Free cancellation until 09.06.2026 11:00",
            "description": "<p>...</p>",
            "fullyRefundable": false
          },
          "remarks": "",
          "roomName":  "Standard Double room (full double bed)",
          "boardName": "Breakfast included"
        }
      ]
    }
  }
}
```


Notes

- `sessionId` — session reference for the cached offers result, used downstream when this offer is acted upon.
- `property` — embedded hotel content: name, star rating, address, images, description, phone, amenities, check-in / check-out times, and latitude / longitude. No separate content fetch is required.
- `offers[<roomName>].rates[].id` identifies the chosen rate.
- `cancelPolicy.rules[]` is normalized to `[{deadline, type, value}]` — the same shape regardless of auth tier.

The endpoint is served from layered 15-minute caches with a fallback to the live supplier query. Passing the same `location` / `checkin` / `checkout` / `guests` as in the prior `stream` search maximizes cache hits.

6. Search — Flights

```
POST /v1/search/flights              -> kick off async search, returns cacheId
GET  /v1/search/flights/{cacheId}    -> read the cached result page
```

Cache TTL: **15 minutes**, identical to hotels.

6.1 Create flight search

```
POST /v1/search/flights
```

Two route shapes are supported. Pick one:

- **Simple round-trip / one-way** — pass `locationFrom`, `locationTo`, `date`, and optionally `dateEnd`.
- **Multi-leg / complex route** — pass `routes[]` with `routeDates[]`. Each `routes[i]` carries its own `locationFrom` / `locationTo`; `routeDates[i]` is the corresponding departure date.

Parameter	Type	Req	Description
`locationFrom`	string	simple	IATA airport code or city code (e.g. `LON`, `JFK`).
`locationTo`	string	simple	Same as above.
`date`	string	simple	Outbound date in `DD.MM.YYYY`.
`dateEnd`	string	no	Return date in `DD.MM.YYYY` (omit for one-way).
`routes`	object[]	complex	`[{locationFrom, locationTo}, ...]`.
`routeDates`	string[]	complex	`["DD.MM.YYYY", ...]` — same length as `routes`.
`cabinClass`	string	yes	One of `economy`, `premiumEconomy`, `business`, `first`.
`adults`	int	yes	1–9.
`children`	int	yes	0–9 (ages 2–11).
`infants`	int	yes	0–9 (under 2).
`currency`	string	no	ISO currency override (e.g. `GBP`, `EUR`, `USD`).
`airlines`	string[]	no	Restrict to a set of marketing carriers (IATA codes).
`onlyCharter`	int	no	`1` to limit to charter inventory.

Response

```
{
  "cacheId": "f9c84b32-7d2f-4a3e-93bd-...",
  "status":  "pending",
  "etaMs":   12000
}
```

The `cacheId` is opaque — pass it as-is to `GET /v1/search/flights/{cacheId}`.

6.2 Read flight search result

```
GET /v1/search/flights/{cacheId}
```

Returns the cached search result. Poll until `status: "completed"`; partial results are returned with `status: "pending"`.


```
{
  "cacheId":  "f9c84b32-...",
  "status":   "completed",
  "currency": "GBP",
  "offers": [
    {
      "id":        "offer-1...",
      "price":     {"total": 412.45, "currency": "GBP", "tax": 142.10},
      "refundable": false,
      "carrier":   "BA",
      "itineraries": [
        {
          "duration": 480,
          "segments": [
            {
              "carrier":   "BA",
              "flightNo":  "175",
              "departure": {"airport": "LHR", "at": "2026-06-11T08:30:00"},
              "arrival":   {"airport": "JFK", "at": "2026-06-11T11:25:00"},
              "cabin":     "economy"
            }
          ]
        }
      ]
    }
  ]
}
```


`offers[].id` is the value to pass downstream as `offerId` when creating an order.

6.3 Calendar search

```
POST /v1/search/flights/calendar
GET  /v1/search/flights/calendar/{sessionId}
```

Returns the lowest available fare per day across a window. Useful for “flexible date” UIs. Body fields are a subset of §6.1 plus a `period` (number of days).

6.4 Comparison table

```
POST /v1/search/flights/comparison-table
GET  /v1/search/flights/comparison-table/{sessionId}
```

Returns the same itinerary across all available fare families (Light / Standard / Flex / Business / etc.) so users can pick a class. Body fields match §6.1; `offerId` is required to anchor the table.

7. Search — Trains

```
POST /v1/search/trains              -> kick off search
GET  /v1/search/trains/{cacheId}    -> read result
```

Coverage: RailEurope (Western Europe), Distribusion (open-data carriers), Onelya / Travelline (CIS), and several regional partners.

7.1 Create train search

```
POST /v1/search/trains
```

Parameter	Type	Req	Description
`route`	int	yes	`1` one-way, `2` round-trip.
`locationFrom`	string	yes	Station code (resolved via `/v1/data/railway-stations`).
`locationTo`	string	yes	Same.
`date`	string	yes	Outbound date `DD.MM.YYYY`.
`timeTo`	string	yes	Outbound earliest time `HH:MM`.
`dateTo`	string	round-trip	Return date `DD.MM.YYYY`.
`timeReturn`	string	round-trip	Return earliest time `HH:MM`.
`adults`	int	yes	1–9.
`children`	int	yes	0–9.
`childAges`	int[]	when `children > 0`	Per-child age, 0–17.
`seniors`	int	no	60+ passengers.
`cabinClass`	string	no	`economy` (default), `business`, `first`.
`service`	string[]	no	Restrict to specific carriers.
`guests`	object[]	yes	Per-coach occupancy (mirrors hotel `guests` shape).

Response

```
{
  "cacheId": "tr-9f2e...",
  "status":  "pending"
}
```

7.2 Read train search result

```
GET /v1/search/trains/{cacheId}
```

Returns offers grouped by carrier, with seat classes, fares, and refundability flags. Each offer has an `id` used as `offerId` on order creation.

8. Search — Transfers

```
POST /v1/search/transfers
```

Synchronous (no cache step) — returns matching transfer offers immediately.

Parameter	Type	Req	Description
`route`	int	yes	`1` one-way, `2` round-trip.
`locationFrom`	string	yes	Place ID (Google Places token resolved via `/v1/data/locations-autocomplete`).
`locationTo`	string	yes	Same.
`locationFromText`	string	yes	Human-readable address for `locationFrom`.
`locationToText`	string	yes	Same.
`locationFromToken`	string	yes	Google session token used at autocomplete time.
`locationToToken`	string	yes	Same.
`dateTo`	string	yes	Pickup date `DD.MM.YYYY`.
`timeTo`	string	yes	Pickup time `HH:MM`.
`dateReturn`	string	round-trip	Return date `DD.MM.YYYY`.
`timeReturn`	string	round-trip	Return time `HH:MM`.
`adults`	int	yes	1–8.
`children`	int	yes	0–6.

Response

```
{
  "offers": [
    {
      "id":         "iway-...",
      "category":   "economy",
      "vehicle":    {"make": "Skoda", "model": "Octavia", "capacity": 3},
      "price":      {"total": 35.00, "currency": "GBP"},
      "supplier":   "iway",
      "cancelable": true
    }
  ]
}
```


9. Orders

A single set of order endpoints handles every service type — flights, hotels, trains, transfers, and combined itineraries. The shape of `params` / `services` varies per type; the lifecycle endpoints are the same.

```
POST   /v1/orders                     -> create
GET    /v1/orders                     -> list (with filters)
GET    /v1/orders/{id}                -> details
GET    /v1/orders/{id}/cancel/check   -> compute penalty preview
POST   /v1/orders/{id}/cancel         -> cancel
GET    /v1/orders/{id}/modify/check   -> validate proposed change
POST   /v1/orders/{id}/modify         -> apply change
```

All order endpoints require auth (JWT or OAuth bearer).

9.1 Create order

```
POST /v1/orders
```

Books a service from a search result. The booking is finalized asynchronously by our queue workers — the response returns immediately with the order in `pre_payment` state; the final status is reachable via `GET /v1/orders/{id}`.

Parameter	Type	Req	Description
`serviceType`	string	yes	One of `flight`, `hotel`, `train`, `transfer`.
`sessionId`	string	yes	`cacheId` (flights/trains) or `sessionId` (hotels) returned by the corresponding search step.
`offerId`	string / int	yes	Offer or rate identifier from the search response. For hotels — `offers[<roomName>].rates[].id`. For flights/trains — `offers[].id`.
`passengers`	object[]	flight/train	Passenger details — first/last name, DOB, gender, document.
`rooms`	object[]	hotel	Per-room guest list: `[{ guests: [{firstName, lastName, ...}], specialRequest: "" }]`. Guest dates use `YYYY-MM-DD`.
`paymentMethod`	string	no	Override default — `card`, `deposit`, `bill`, etc. Defaults vary per account.
`bookKey`	string	no	Hotel-only: idempotency hint for repeated price-check confirmations.
`Idempotency-Key` (header)	string	no	Prevents duplicate orders on retries. Same key within 24 h returns the prior order.

Response

```
{
  "order": {
    "id":          57975,
    "code":        "GB260611005",
    "status":      "pre_payment",
    "currency":    "GBP",
    "priceGross": 412.45,
    "payPrice":    412.45,
    "paid":        0,
    "tourBegin":   "2026-06-11",
    "tourEnd":     "2026-06-11",
    "services":    [ /* ... */ ],
    "clients":     [ /* passengers */ ]
  }
}
```

For hotels that pass live price-checking, the same response is returned. If the supplier's price has drifted since search, the response is `409 Conflict` with the new price-check payload — call `POST /v1/orders` again with the same `bookKey` to confirm at the new price.

9.2 List orders

```
GET /v1/orders?status=finish&dateFrom=2026-04-01&limit=20&offset=0
```

Parameter	Type	Description
`code`	string	Order code (exact match).
`status`	string / string[]	One or many of `pre_order`, `pre_payment`, `pre_book`, `finish`, `canceled`.
`pnr`	string	Air PNR.
`passengerName`	string	Substring match on passenger last name.
`paymentStatus`	string / string[]	Payment lifecycle filter.
`dateFrom` / `dateTo`	string	Order creation date window, `YYYY-MM-DD`.
`tourBeginFrom` / `tourBeginTo`	string	Departure date window.
`tourEndFrom` / `tourEndTo`	string	Return date window.
`agencyId`	int / int[]	Restrict to a sub-agency (corp accounts).
`sort` / `sortOrder`	string	E.g. `sort=createdAt&sortOrder=desc`.
`offset` / `limit`	int	Pagination.

9.3 Get one

```
GET /v1/orders/{id}
```

Returns the full order including services, passengers, payments, and (when applicable) rewards/loyalty annotations.

9.4 Cancel — preview penalty

```
GET /v1/orders/{id}/cancel/check
```

Returns whether the order can be cancelled and what penalty applies, **without** acting:

```
{
  "cancellable":      true,
  "refundAmount":     180.20,
  "penaltyAmount":    32.25,
  "currency":         "GBP",
  "details": [
    {
      "serviceId":   1234,
      "type":        "hotel",
      "title":       "Standard Double Room — Hotel X",
      "refundable":  true,
      "deadline":    "2026-06-09T11:00:00+00:00",
      "penalty":     0
    }
  ]
}
```


Always call `cancel/check` before showing a confirmation prompt — penalty calculation may involve a live supplier query.

9.5 Cancel

```
POST /v1/orders/{id}/cancel
```

Parameter	Type	Req	Description
`reason`	string	no	Free text — surfaced in the order audit trail.

Triggers the cancellation flow. Refunds are credited to the original payment method (or to deposit balance for non-card flows). The response is the updated order.

9.6 Modify — preview change

```
GET /v1/orders/{id}/modify/check?type=<type>&serviceId=<id>
```

Returns whether a specific change is allowed and what it would cost.

`type` is one of:

Type	Effect
`contact`	Update passenger email/phone (free).
`passport`	Update passenger document number / expiry (carrier-dependent fee).
`rebook`	Change date/route — quotes the new fare difference.
`baggage`	Add additional baggage allowance (carrier-dependent).

9.7 Modify — apply

```
POST /v1/orders/{id}/modify
```

Parameter	Type	Req	Description
`type`	string	yes	Same as 9.6.
`changes`	object	yes	Type-specific payload. Always includes `serviceId`.

Successful modifications return the updated order. For `rebook` the response also includes the new fare and any debt/credit applied.

10. Wallet (deposit)

For corp accounts that pre-fund a deposit balance:

Endpoint	Method	Purpose
`/v1/wallet`	GET	Current balance per currency.
`/v1/wallet/transactions`	GET	Paginated history of top-ups, charges, refunds.
`/v1/wallet/top-up/bill`	POST	Issue an invoice for bank transfer.
`/v1/wallet/top-up/card`	POST	Charge a saved card to top up the balance.
`/v1/wallet/top-up/crypto`	POST	Generate a crypto payment address.
`/v1/wallet/notification`	PUT	Configure low-balance alerts.

Bookings paid with `paymentMethod: "deposit"` deduct from this balance.

11. Quick examples

11.1 Authenticate

```
curl -s -X POST "https://api.travel-code.com/v1/auth/login" \
  -H "Content-Type: application/json" \
  -d '{"email": "ops@example.com", "password": "***"}'
# → {"accessToken": "eyJ...", "expiresAt": "..."}

TOKEN="eyJ..."
```

11.2 Refresh tokens before expiry

```
curl -s -X POST "https://api.travel-code.com/v1/auth/jwt/refresh" \
  -H "Content-Type: application/json" \
  -d '{"refreshToken": "eyJ..."}'
```

11.3 Resolve a hotel location

```
curl -s "https://api.travel-code.com/v1/data/hotel-locations?search=Tbilisi&limit=5" | jq .
```

11.4 Stream a hotel search

```
curl -s -N -X POST "https://api.travel-code.com/v1/search/hotels/stream" \
  -H "Content-Type: application/json" \
  -d '{
    "location": 12345,
    "checkin":  "2026-04-15",
    "checkout": "2026-04-18",
    "countryCode": "UK",
    "guests": [{"adults": 2, "children": 1, "childrenAges": [5]}],
    "offset": 0, "limit": 20,
    "sort": "recommend",
    "filter": {"starRating[]": [4, 5], "boards[]": ["AI", "FB"], "payments": ["full_refund"]}
  }' --max-time 130
```

11.5 Get rates for one hotel

```
curl -s -X POST "https://api.travel-code.com/v1/search/hotels/offers" \
  -H "Content-Type: application/json" \
  -d '{
    "id": 1431950,
    "checkin":  "2026-04-15",
    "checkout": "2026-04-18",
    "countryCode": "UK",
    "guests": [{"adults": 2}],
    "location": 12345
  }' | jq '.offers | to_entries[0]'
```

11.6 List from cache (paginate / restrict by id)

```
curl -s -X POST "https://api.travel-code.com/v1/search/hotels/list" \
  -H "Content-Type: application/json" \
  -d '{
    "location": 12345,
    "checkin":  "2026-04-15",
    "checkout": "2026-04-18",
    "countryCode": "UK",
    "guests": [{"adults": 2}],
    "id": [1431950, 982412],
    "offset": 0, "limit": 20
  }' | jq .
```

11.7 Map (geo points, viewport-scoped)

```
curl -s -X POST "https://api.travel-code.com/v1/search/hotels/map" \
  -H "Content-Type: application/json" \
  -d '{
    "location": 12345,
    "checkin":  "2026-04-15",
    "checkout": "2026-04-18",
    "countryCode": "UK",
    "guests": [{"adults": 2}],
    "mapBoundsSwLat": 41.3431, "mapBoundsSwLng": 2.0695,
    "mapBoundsNeLat": 41.4696, "mapBoundsNeLng": 2.2280
  }' | jq .
```

11.8 Search by hotel name within cache

```
curl -s "https://api.travel-code.com/v1/search/hotels/search-names?ck=$CK&q=pullman" | jq .
```

11.9 Search flights → poll → book

```
# 1. Kick off search
CACHE=$(curl -s -X POST "https://api.travel-code.com/v1/search/flights" \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "locationFrom": "LON",
    "locationTo":   "JFK",
    "date":         "11.06.2026",
    "dateEnd":      "18.06.2026",
    "cabinClass":   "economy",
    "adults": 1, "children": 0, "infants": 0
  }' | jq -r .cacheId)

# 2. Poll
OFFER=$(curl -s "https://api.travel-code.com/v1/search/flights/$CACHE" \
  -H "Authorization: Bearer $TOKEN" | jq -r '.offers[0].id')

# 3. Book
curl -s -X POST "https://api.travel-code.com/v1/orders" \
  -H "Authorization: Bearer $TOKEN" \
  -H "Idempotency-Key: $(uuidgen)" \
  -H "Content-Type: application/json" \
  -d "{
    \"serviceType\": \"flight\",
    \"sessionId\":   \"$CACHE\",
    \"offerId\":     \"$OFFER\",
    \"passengers\": [{
      \"firstName\": \"Alex\", \"lastName\": \"Doe\",
      \"birthDate\": \"1990-05-12\", \"gender\": \"M\",
      \"document\": {\"type\": \"passport\", \"number\": \"X12345\", \"expiry\": \"2030-01-01\"}
    }]
  }"
```

11.10 Book a hotel rate

```
curl -s -X POST "https://api.travel-code.com/v1/orders" \
  -H "Authorization: Bearer $TOKEN" \
  -H "Idempotency-Key: $(uuidgen)" \
  -H "Content-Type: application/json" \
  -d '{
    "serviceType": "hotel",
    "sessionId":   "69f38317223480.46921054",
    "offerId":     "5e266508-dee21a18-6599dbce-9dabfbe3",
    "rooms": [{
      "guests": [
        {"firstName": "Alex", "lastName": "Doe", "birthDate": "1990-05-12", "isMain": true},
        {"firstName": "Sam",  "lastName": "Doe", "birthDate": "1992-09-04"}
      ],
      "specialRequest": "Quiet room, high floor"
    }]
  }'
```

11.11 Cancel with penalty preview

```
ORDER_ID=57975

# 1. Preview penalty
curl -s "https://api.travel-code.com/v1/orders/$ORDER_ID/cancel/check" \
  -H "Authorization: Bearer $TOKEN" | jq .

# 2. Confirm
curl -s -X POST "https://api.travel-code.com/v1/orders/$ORDER_ID/cancel" \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"reason": "Customer requested change of plans"}'
```

11.12 Wallet balance

```
curl -s "https://api.travel-code.com/v1/wallet" \
  -H "Authorization: Bearer $TOKEN" | jq .
```

Appendix A — Error envelope

All non-2xx responses share a single shape:

```
{
  "error": {
    "code":    "OFFER_EXPIRED",
    "message": "The chosen offer is no longer available. Please re-run the search.",
    "details": null
  }
}
```

Common codes integrators should handle:

Code	HTTP	Meaning
`TOKEN_NOT_PROVIDED`	401	Missing `Authorization` header.
`TOKEN_EXPIRED`	401	Refresh required.
`CACHE_NOT_FOUND`	400	The provided `cacheId` / `sessionId` does not exist or has expired (15 minutes). Re-run the search step.
`OFFER_EXPIRED`	400	Offer / rate is no longer bookable — re-run the search.
`FORM_VALIDATION_ERROR`	400	Request body failed validation; details in `error.details`.
`ORDER_NOT_FOUND`	404	Order does not exist or belongs to a different account.
`ORDER_NOT_MODIFIABLE`	400	Order status does not allow the requested change/cancel.
`MODIFICATION_UNAVAILABLE`	400	Specific modification type not supported for this service/carrier.
`LOCATION_NOT_FOUND`	404	Location code unknown — resolve via `/v1/data/...` lookups.

Appendix B — Versioning & support

- This document describes API version **v1**. Breaking changes will be released under `v2` with a 6-month deprecation window.
- Partner onboarding, support, and test credentials are provided by your account manager at sign-up.